



Multimodal AI Agent for Customer Care

Project Work Digital Business - TP Italia x Politecnico di Bari

BENCHMARKING TECNOLOGICO

SCENARIO APPLICATIVO

Chatbot AI multimodale per assistenza a cittadini, turisti e visitatori durante i Giochi del Mediterraneo Taranto 2026

DESCRIZIONE

Lo scenario applicativo selezionato riguarda la realizzazione di un assistente AI multimodale verticalizzato sulla città di Taranto e sui Giochi del Mediterraneo Taranto 2026. Il sistema nasce come punto di accesso conversazionale unico per cittadini, turisti, spettatori, volontari, operatori e visitatori che, durante il periodo precedente e durante lo svolgimento dell'evento, avranno bisogno di orientarsi tra informazioni sportive, logistiche, turistiche e operative distribuite su molteplici fonti.

L'obiettivo principale è supportare l'utente nell'accesso rapido e personalizzato alle informazioni relative ai Giochi del Mediterraneo 2026, con particolare attenzione a venue, discipline sportive, calendario, città coinvolte, indicazioni di accesso agli impianti, ticketing, volontariato, partnership, progetto scuole, contatti ufficiali e informazioni storiche sui Giochi.

Il progetto nasce a causa della frammentazione delle informazioni legate a un grande evento sportivo internazionale, poco conosciuto nonostante l'ampia rilevanza. Le informazioni su sport, impianti, calendario, biglietti, volontari, bandi, contatti, accessibilità, mobilità, luoghi di gara e storia dei Giochi sono distribuite tra sito ufficiale, pagine informative, documenti PDF, archivi storici, mappe e fonti eterogenee. Per un visitatore o un cittadino, recuperare rapidamente la risposta corretta può essere complesso, soprattutto se la domanda è formulata in modo naturale, incompleto o non tecnico.

Il sistema è pensato come agente AI multimodale capace di interpretare input testuali, vocali e immagini.

Il progetto ha l'obiettivo di assorbire richieste informative di primo livello e ridurre il carico sui canali di assistenza. Le domande semplici e ricorrenti possono essere gestite automaticamente, mentre le richieste non coperte dalla knowledge base o quelle che richiedono dati live possono essere indirizzate verso un operatore.

ARCHITETTURA

Il principio architetturale è separare quattro livelli principali: il livello di interazione utente, il livello di backend, il livello di conoscenza RAG e il livello operativo per ticketing ed escalation.

Il livello di interazione utente comprende l'interfaccia attraverso cui cittadini, turisti e visitatori dialogano con il sistema, mediante una web chat accessibile da browser in cui l'utente pone domande in linguaggio naturale e riceve risposte discorsive con fonti, eventuali link Maps e indicazioni operative. Il frontend può integrare input vocali, caricamento immagini, suggerimenti, visualizzazione di fonti e collegamenti alla navigazione.

Il livello di backend gestisce la ricezione delle domande, la normalizzazione della query, l'interrogazione del vector database, la costruzione del contesto RAG, la generazione della risposta e l'eventuale post-processing.

Il livello di conoscenza RAG è basato su una knowledge base. Ogni record contiene un identificativo univoco, un documento testuale e metadata.

Il livello operativo riguarda il monitoraggio delle conversazioni, la raccolta di feedback, la valutazione delle metriche di performance e l'apertura di ticket verso operatori umani.

La scelta delle tecnologie privilegia soluzioni gratuite, senza dipendenza da API commerciali per embedding, generazione o ticketing. Il costo non viene sostenuto a token, ma dipende dalle risorse computazionali locali o cloud utilizzate per eseguire i servizi. La containerizzazione tramite Docker permette di rendere il sistema replicabile e facilmente distribuibile.

BENCHMARKING

Frontend Utente

Il frontend deve permettere all'utente di porre domande in linguaggio naturale, caricare immagini, inviare audio e ricevere risposte articolate.

Le opzioni considerate sono React, Streamlit e Gradio.

Tecnologia	Vantaggi	Limiti	Valutazione business
React	Alta flessibilità e ampia documentazione, adatto a interfacce web progressive e responsive	Richiede più sviluppo rispetto a Streamlit	Migliore scelta per un prodotto robusto e scalabile con chat e dashboard operatore
Streamlit	Rapido per prototipi AI e data-driven, semplice in Python	Meno adatto a UI con gestione avanzata dello stato	Ottimo per demo e prototipo interno
Gradio	Molto rapido per demo multimodali	Meno adatto a dashboard operatore e prodotto finale	Utile per testare modelli, meno per web app completa

React è la scelta migliore per il frontend utente. La documentazione ufficiale lo presenta come libreria per costruire interfacce componibili a partire da componenti riutilizzabili, caratteristica utile per separare chat, elementi di risposta, upload file, suggerimenti, feedback e dashboard operatore.

Backend e API

Il backend deve esporre API per chat, caricamento documenti, retrieval, generazione risposte, apertura ticket, logging e dashboard. Le tecnologie considerate sono FastAPI, Flask e Django, framework Python, in quanto tale linguaggio di programmazione è il più adatto e utilizzato in contesti AI/RAG/LLM.

Tecnologia	Vantaggi	Limiti	Valutazione business
FastAPI	API moderne, performance elevate, tipizzazione Python, documentazione OpenAPI automatica	Richiede strutturazione manuale dei moduli	Migliore equilibrio tra velocità, pulizia e integrazione AI

Flask	Semplice e leggero	Meno strutturato per API complesse e validazione dati	Buono per micro-prototipi
Django	Completo, include ORM e admin	Più pesante per un backend AI modulare	Utile per web app pura

Python + FastAPI è la scelta più coerente perché consente di costruire rapidamente API efficienti e con documentazione automatica, fondamentale in ambito business e con team eterogenei.

Knowledge Base e Data Layer

La Knowledge Base del progetto contiene i dati sulla quale si basa la risposta generata dal modello di linguaggio.

Tecnologia	Vantaggi	Limiti	Valutazione business
File JSON/CSV normalizzati	Semplici, gratuiti, versionabili, adatti al prototipo	Meno adatti a gestione multiutente e storico modifiche	Ottimi per fase iniziale
PostgreSQL	Robusto, query strutturate, supporto JSONB, indicizzazione	Richiede setup DB	Migliore scelta per dati strutturati e logging
SQLite	Semplice, serverless, gratuito	Meno adatto a dashboard multiutente e concorrenza	Buono per prototipo locale minimo

Per la persistenza dei dati prima della fase di ingest per il RAG, il formato JSONL è il più adatto, poiché permette di conservare singoli record compatti e atomici in file in formato JSON. PostgreSQL è consigliato come database relazionale di supporto per utenti, sessioni, feedback, ticket, log e dati trasversali per la Knowledge Base.

Embedding

Il modello di embedding trasforma i documenti in vettori semantici interrogabili.

Tecnologia	Vantaggi	Limiti	Valutazione business
BAAI/bge-m3	Multilingua, supporta dense retrieval, sparse retrieval e multi-vector retrieval, input lunghi fino a 8192 token	Più pesante di MiniLM	Migliore scelta per qualità RAG gratuita
multilingual-e5-large	Forte per retrieval multilingua	Meno flessibile sul piano multi-retrieval rispetto a BGE-M3	Alternativa solida
paraphrase-multilingual-MiniLM	Molto leggero e rapido	Qualità inferiore su retrieval complesso	Buono per PC poco potenti
Qwen3-Embedding	Modelli moderni, multilingua, varie dimensioni	Più recente e potenzialmente più pesante	Interessante per confronto sperimentale

BGE-M3 è la scelta consigliata, poiché nativamente multilingua (oltre 100 lingue) e supporta input fino a 8192 token.

Vector Database

Il vector database deve indicizzare gli embedding e supportare ricerca semantica con eventuali filtri.

Tecnologia	Vantaggi	Limiti	Valutazione business
Qdrant	Vector database dedicato, self-hosted, supporta payload e filtri, adatto a servizi backend	Più strutturato di Chroma	Migliore scelta per sistema robusto

ChromaDB	Semplice per MVP RAG, integra embedding e metadata	Meno orientato a separazione backend/database	Ottimo per prima versione del progetto
FAISS	Molto efficiente per similarity search	Non è un database completo, non offre gestione metadata e necessita di API da costruire	Utile per ricerca vettoriale pura
Milvus	Molto scalabile, adatto a dataset grandi	Più complesso per il progetto	Eccessivo per ChatBot AI

ChromaDB è la scelta migliore, poiché consente di memorizzare embedding con metadati, effettuare ricerca su vettori dense e sparse, filtrare tramite metadati e recuperare contenuti testuali o multimodali. La documentazione ufficiale lo presenta esplicitamente come soluzione che fornisce ciò che serve per il retrieval in applicazioni AI.

Large Language Model (LLM)

Il modello linguistico deve generare risposte naturali, sintetiche, utili e vincolate al contesto recuperato dal RAG. Il vincolo è mantenere la soluzione gratuita e replicabile.

Tecnologia	Vantaggi	Limiti	Valutazione business
Qwen3 8B via Ollama	Multilingua, locale, gratuito, semplice da servire	Richiede risorse hardware adeguate	Migliore equilibrio qualità/semplificà
Llama 3.1/3.2 8B via Ollama	Molto diffuso, buona qualità generale	Italiano e contesto urbano da verificare sperimentalmente	Alternativa solida
Mistral 7B	Leggero e veloce	Qualità variabile su italiano e istruzioni complesse	Buono se l'hardware è limitato
vLLM + modello open-weight	Più performante e production-oriented	Più complesso, richiede GPU/server	Utile per progetto enterprise robusto

Qwen3 8B è consigliato per il prototipo avanzato perché supporta oltre 100 lingue e dialetti. Ollama è consigliato come motore locale perché espone API per interagire con i modelli e consente di eseguirli in locale con una complessità operativa bassa.

Assunte le criticità relative alle risorse computazionali a disposizione dei membri del team per lo sviluppo, si lascia aperta la possibilità di integrare modelli LLM a chiamate API via OpenAI, come Groq con Llama. Tale alternativa permetterebbe di ottenere risposte veloci e robuste, ma con un limite di richieste prefissato (il limite è comunque alto e perfettamente compatibile con gli obiettivi del progetto e le previsioni di richieste di test/demo).

Pipeline Multimodale (Audio e Immagini)

Lo scenario prevede input testuali, immagini e audio. Per mantenere la soluzione gratuita, la strategia consigliata è convertire audio e immagini in testo prima della fase RAG. L'audio viene trascritto, le immagini vengono processate con OCR per estrarre contenuto testuale e/o vengono generati embedding visivi per riconoscimenti venue e luoghi di interesse. Il testo unico risultante viene inviato alla stessa pipeline di retrieval.

Componente	Tecnologia	Criteri di valutazione
Audio-to-text	Whisper, Vosk	Robustezza multilingua e qualità generale
OCR immagini/documenti	PaddleOCR, Tesseract, EasyOCR	Copertura su scenari eterogenei e documenti
Vision-language model	Moondream, Qwen3-VL, LLaVA	Costo (complessità)-opportunità per il progetto

Whisper è un modello open source per speech recognition multilingua, traduzione e language identification, quindi coerente con un assistente urbano usato da cittadini e turisti.

Per l'OCR, Tesseract è preferibile a PaddleOCR se si vuole gestire testo in immagini, documenti o contesti visivi vari ma senza complessità aggiuntiva, con il limite di foto pulite e nitide, riducendo i tempi di inferenza e risposta.

Per il riconoscimento di monumenti e luoghi di interesse da immagine viene utilizzato Moondream, un modello di linguaggio visuale che codifica le immagini in rappresentazioni numeriche e le combina con il prompt testuale al fine di generare una risposta testuale. Le skill dichiarate sono object detection, pointing/counting, visual question answering e captioning.

Dashboard Operatore

La dashboard operatore deve mostrare richieste gestite, ticket aperti, feedback utenti e KPI di qualità.

Tecnologia	Vantaggi	Limiti	Valutazione business
React	Coerente con frontend, massima personalizzazione	Più sviluppo	Migliore per dashboard integrata nel prodotto
Streamlit	Rapida da costruire, ottima per KPI e prototipo	Meno enterprise	Migliore per prima demo interna
Grafana	Forte per metriche tecniche e osservabilità	Meno adatta a ticketing custom	Ideale per monitoraggio tecnico

La scelta consigliata risulta la dashboard in React.

Versioning e Deployment

La strategia di versioning e deploy deve garantire quattro obiettivi principali: riproducibilità dell'ambiente, tracciabilità delle modifiche e facilità di collaborazione nel team.

Le principali alternative valutate sono GitHub, GitLab e Bitbucket. Tutte supportano repository Git, collaborazione tramite branch e pull/merge request, ma differiscono per semplicità e integrazione CI/CD.

Il deploy deve essere valutato rispetto al sistema RAG con backend FastAPI, frontend React, PostgreSQL, ChromaDB e LLM locale tramite Ollama. Questo rende sensato un deployment containerizzato.

Soluzione	Pro	Contro	Valutazione
Docker Compose locale	Gratuito, riproducibile, adatto a backend + DB + ChromaDB + frontend	Non è produzione cloud reale	Scelta migliore per MVP
Vercel per frontend + backend locale/cloud separato	Ottimo per React/Vite, deploy automatico da GitHub	Backend AI locale resta da esporre separatamente	Buono solo per frontend

Render/Railway/Fly.io	Possibili deploy cloud più semplici di Kubernetes	Free tier, limiti risorse, LLM locale problematico	Possibile ma rischioso
Kubernetes	Standard enterprise per orchestrazione container	Eccessivo per MVP, curva alta	Non consigliato
Deployment manuale senza Docker	Facile inizialmente	Poco riproducibile	Da evitare

RIFERIMENTI

<https://react.dev>

<https://fastapi.tiangolo.com>

<https://www.postgresql.org/docs/current/datatype-json.html>

<https://huggingface.co/BAAI/bge-m3>

<https://docs.trychroma.com/docs/overview/introduction>

<https://huggingface.co/Qwen/Qwen3-8B>

<https://docs.ollama.com/api/introduction>

<https://github.com/openai/whisper>

<https://paddlepaddle.github.io/PaddleOCR/main/en/index.html>

<https://tesseract-ocr.github.io>

<https://docs.moondream.ai/>

<https://docs.streamlit.io>